

PROJECT REPORT

SecureVault Campus

A Simulated PAM Lab Using CyberArk Conjur, Windows Server Active Directory, LAPS, and JEA
CyberArk PAM Engineer Portfolio

Shyam Kumar D

Aspiring Cloud Infrastructure Engineer | AWS Cloud, Networking & Security

linkedin.com/in/shyam-kumar-d | github.com/ShyamD2

July 2026

1 Executive Summary

SecureVault Campus is a self-built Privileged Access Management (PAM) lab that I designed and deployed entirely on my own laptop to demonstrate the core concepts behind CyberArk's PAM suite without access to a paid enterprise license. The lab consists of three virtual machines — a Windows Server 2025 Active Directory Domain Controller (DC01), a domain-joined member server (CLIENT01), and an Ubuntu Server VM running CyberArk Conjur Open Source in Docker (CONJURVM). Together these machines let me build and demonstrate identity management, automated privileged-password rotation, restricted administrative sessions, and secrets management — the same problems CyberArk's commercial products are built to solve.

I built this project because CyberArk's full commercial suite (Vault, PVWA, CPM, PSM) requires a paid enterprise license that isn't available to a student, but the underlying engineering concepts can still be demonstrated honestly using free, real tools. Wherever the guide I followed called for a licensed CyberArk component, I substituted a genuine, functionally equivalent free tool: Microsoft LAPS (Local Administrator Password Solution) in place of CyberArk's CPM for automated local-admin password rotation, and PowerShell JEA (Just Enough Administration) in place of CyberArk's PSM for restricted, least-privilege administrative sessions. For the one piece of the puzzle that CyberArk does publish for free, I used the real thing: CyberArk Conjur Open Source, running policy-as-code secret storage and retrieval exactly as it would in a production environment.

This report documents each stage of the build in the order I actually executed it, with the exact PowerShell/CLI commands I ran and a screenshot proving the result. I have been careful throughout to be upfront about what is a genuine CyberArk product versus a free functional analog — I believe that honesty is a stronger talking point than implying I had access to the full paid suite, and it is the approach I use consistently throughout this report.

2 Architecture Overview

The lab runs three virtual machines under Oracle VirtualBox, connected through a shared NAT Network (SecureVaultNet, 192.168.56.0/24) so they can reach each other and the internet. I ran the Windows VMs together only when a step required them to communicate (domain join, LAPS, JEA), and ran CONJURVM independently to conserve RAM on my 8 GB host laptop.

VM Name	Role	What It Demonstrates	Real CyberArk Component Simulated
DC01	Windows Server 2025 — Active Directory Domain Controller	AD forest setup, DNS, Group Policy, OU structure, user/computer management	The identity backbone every PAM deployment sits on top of
CLIENT01	Windows Server 2025 — domain-joined member server (Server Core)	Domain join, GPO application, LAPS password rotation, JEA restricted sessions	CPM (Central Password Manager) and PSM (Privileged Session Manager)

CONJURVM	Ubuntu Server 24.04 — CyberArk Conjur Open Source in Docker	Secrets storage, policy-as-code, secret retrieval via CLI/API	CyberArk Conjur (the real, free, open-source CyberArk product)
----------	---	--	---

DC01 (192.168.56.10) is the domain controller for securevault.local, hosting Active Directory Domain Services, DNS, and Group Policy Management. CLIENT01 (192.168.56.11) is a domain-joined Server Core member server used to prove out LAPS password rotation and JEA restricted sessions. CONJURVM runs CyberArk's own open-source Conjur secrets engine inside Docker containers, exposed through an Nginx proxy and the official Conjur CLI client container.

3 Step-by-Step Build with Screenshots

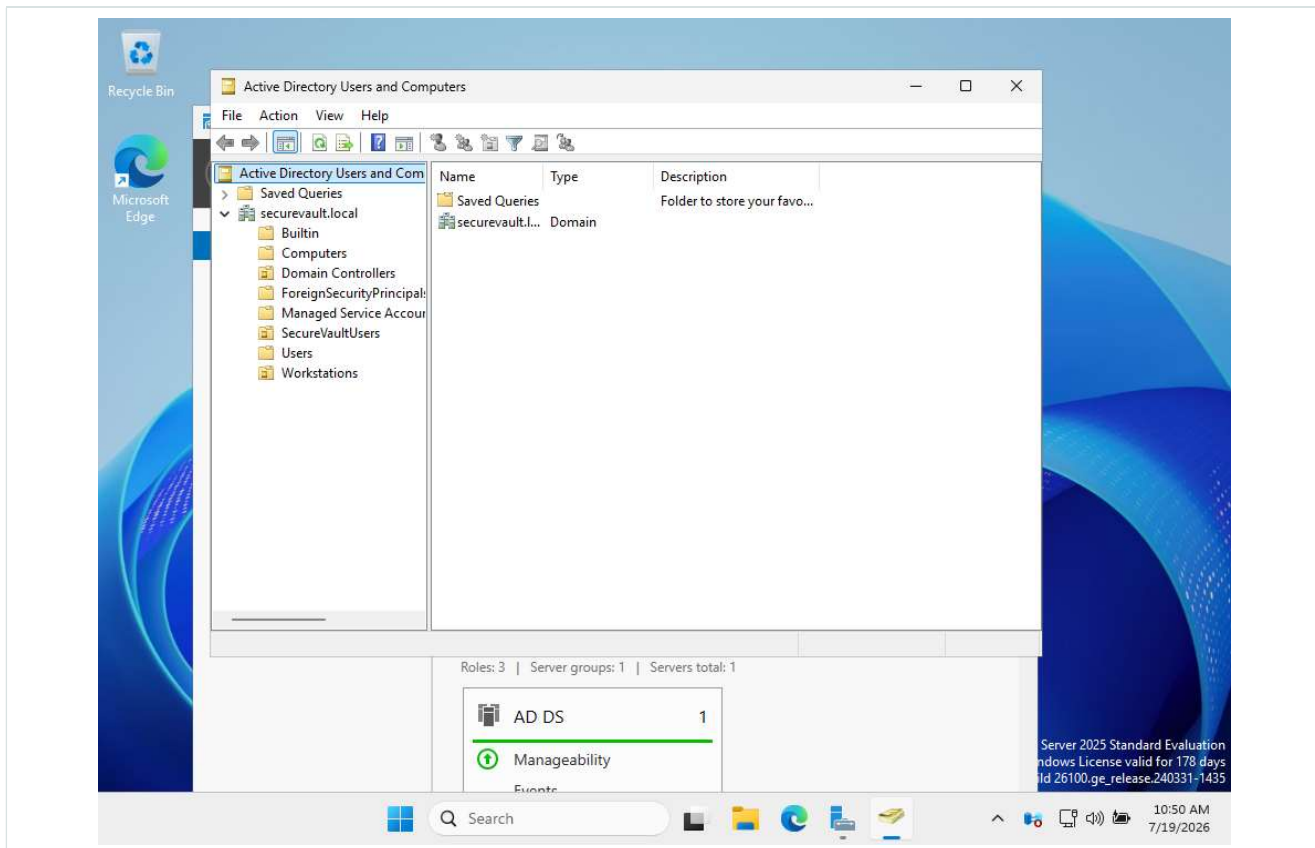
The screenshots below are presented in the order I actually executed the build, following the technical dependency chain of the lab: Active Directory foundation → domain join → LAPS (CPM stand-in) → JEA (PSM stand-in) → Conjur (real CyberArk secrets engine) → PowerShell audit reporting. Passwords, secret values, and API keys visible in the original screenshots have been redacted with black bars before inclusion in this report.

4.7 — Active Directory OU Structure Created

Command(s) executed:

```
New-ADOrganizationalUnit -Name "Workstations" -Path "DC=securevault,DC=local"  
New-ADOrganizationalUnit -Name "SecureVaultUsers" -Path "DC=securevault,DC=local"
```

This screenshot, taken in Active Directory Users and Computers on DC01, shows the securevault.local domain with the SecureVaultUsers and Workstations organizational units in place, alongside the default BuiltIn, Computers, and Users containers. This OU structure is the identity backbone that every PAM deployment — CyberArk's included — sits on top of; without a clean AD hierarchy there is nothing for policy-based tools like LAPS or GPOs to target.



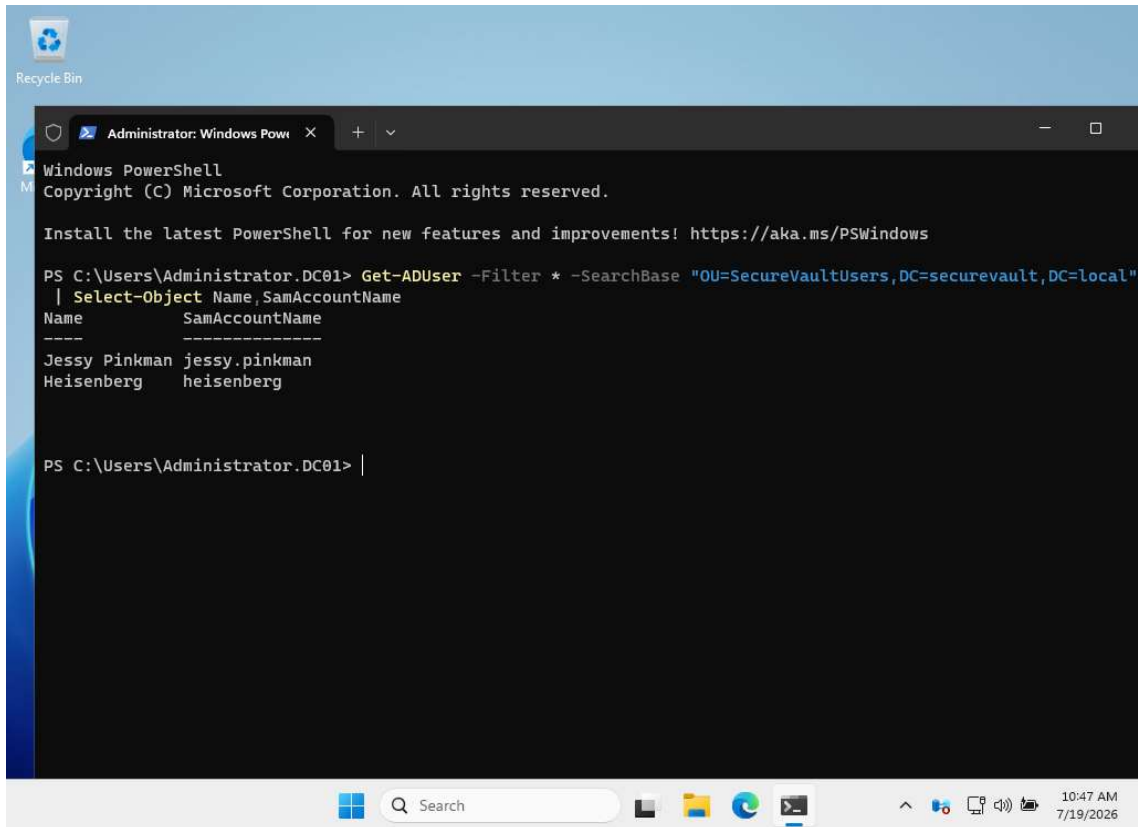
Active Directory Users and Computers — securevault.local domain and OU structure

4.7 — Active Directory Test Users Verified

Command(s) executed:

```
Get-ADUser -Filter * -SearchBase "OU=SecureVaultUsers,DC=securevault,DC=local" |  
Select-Object Name,SamAccountName
```

Running Get-ADUser against the SecureVaultUsers OU confirms both test accounts I created were provisioned correctly with the right SamAccountName values. These accounts stand in for the human identities a real PAM deployment would onboard — one of them (used later for JEA) represents a restricted operator, mirroring how CyberArk PSM assigns scoped, non-administrative access to specific users.



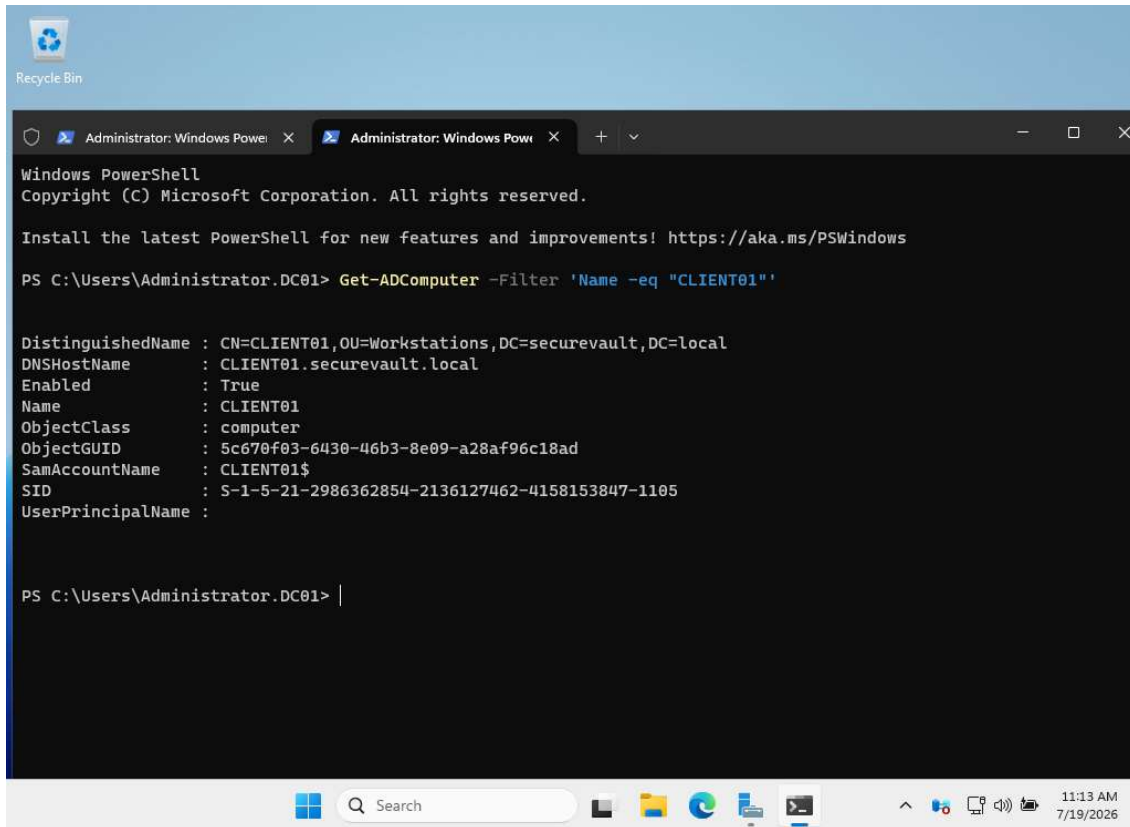
Get-ADUser output confirming the two test accounts in the SecureVaultUsers OU

5.6 — CLIENT01 Domain-Joined and Moved to the Workstations OU

Command(s) executed:

```
Add-Computer -DomainName "securevault.local" -Credential ... -Restart
Move-ADObject -Identity (Get-ADComputer -Filter 'Name -eq "CLIENT01"]').DistinguishedName
-TargetPath "OU=Workstations,DC=securevault,DC=local"
Get-ADComputer -Filter 'Name -eq "CLIENT01"'
```

This confirms CLIENT01's computer object now lives under OU=Workstations in the securevault.local domain, which is a prerequisite for the LAPS Group Policy (scoped to that OU) to apply. This step represents the basic device-onboarding process any PAM tool depends on — a managed endpoint has to be a known, correctly-placed object in the directory before automated credential rotation can target it.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator.DC01> Get-ADComputer -Filter 'Name -eq "CLIENT01"'

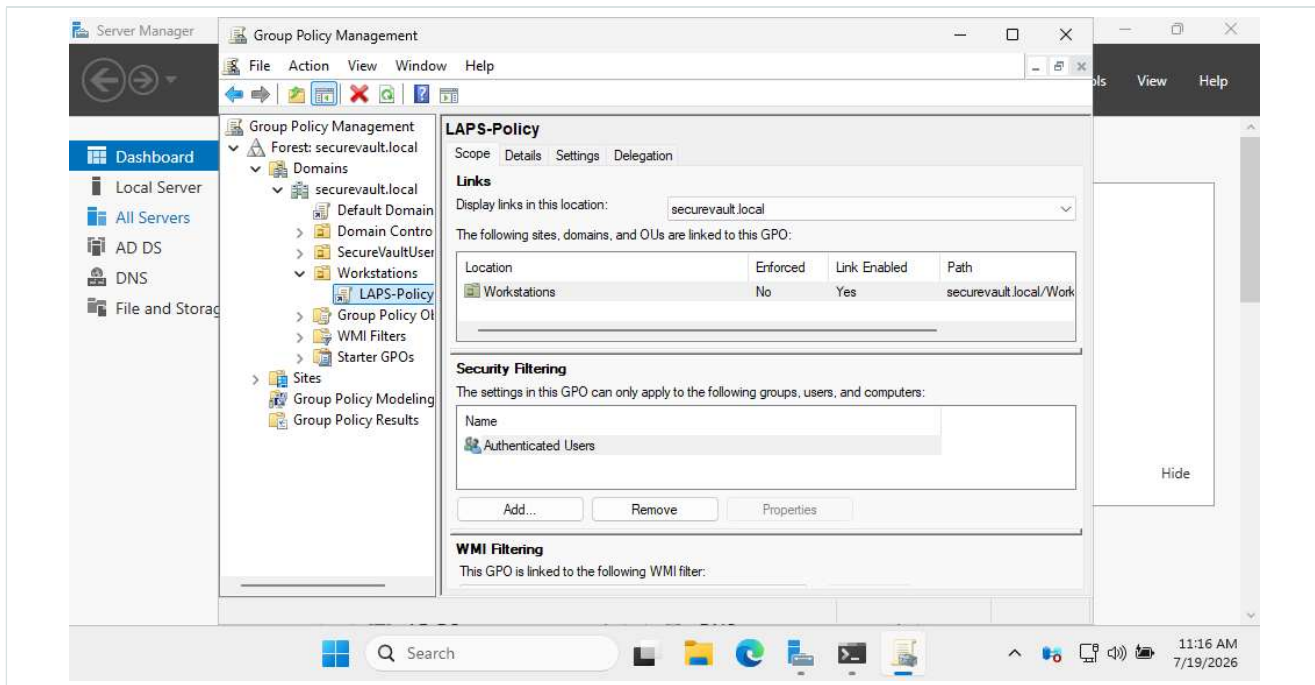
DistinguishedName : CN=CLIENT01,OU=Workstations,DC=securevault,DC=local
DNSHostName       : CLIENT01.securevault.local
Enabled           : True
Name              : CLIENT01
ObjectClass       : computer
ObjectGUID        : 5c670f03-6430-46b3-8e09-a28af96c18ad
SamAccountName    : CLIENT01$
SID               : S-1-5-21-2986362854-2136127462-4158153847-1105
UserPrincipalName :

PS C:\Users\Administrator.DC01> |
```

Get-ADComputer confirming CLIENT01's location in the Workstations OU

6.3 — LAPS Group Policy Configured

This screenshot shows the Group Policy Management console on DC01 with the LAPS-Policy GPO linked to the Workstations OU. Inside this GPO I enabled "Configure password backup directory" (set to Active Directory) and "Password Settings" (8-character complexity, 14-day rotation age). This is the direct free-tool stand-in for CyberArk's CPM (Central Password Manager) — instead of a licensed engine pushing rotation policy from a vault, Windows LAPS pushes the same rotation behavior through native Group Policy.



Group Policy Management — LAPS-Policy GPO linked and scoped to the Workstations OU

6.5 — LAPS-Rotated Password Retrieved

Command(s) executed:

```
Get-ADUser -Filter * -SearchBase "OU=SecureVaultUsers,DC=securevault,DC=local" | Select-Object Name,SamAccountName
Get-LapsADPassword -Identity "CLIENT01" -AsPlainText
```

Get-LapsADPassword pulls back CLIENT01's current local Administrator password directly from its encrypted Active Directory attribute, along with the rotation and expiration timestamps and the authorized decryptor group (SECUREVAULT\Domain Admins). This is the working proof that automated privileged-credential rotation is functioning end-to-end — functionally the same outcome CyberArk's CPM delivers for local admin accounts, just implemented with Microsoft's free, AD-native LAPS engine. The actual password value is redacted in this screenshot.

```

Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator.DC01> Get-ADUser -Filter * -SearchBase "OU=SecureVaultUsers,DC=securevault,DC=local"
| Select-Object Name,SamAccountName
Name           SamAccountName
-----
Jessy Pinkman   jessy.pinkman
Heisenberg      heisenberg

PS C:\Users\Administrator.DC01> Get-LapsADPassword -Identity "CLIENT01" -AsPlainText

ComputerName      : CLIENT01
DistinguishedName : CN=CLIENT01,OU=Workstations,DC=securevault,DC=local
Account           : Administrator
Password          : 
PasswordUpdateTime : 7/20/2026 9:58:27 AM
ExpirationTimestamp : 8/17/2026 9:58:27 AM
Source            : EncryptedPassword
DecryptionStatus   : Success
AuthorizedDecryptor : SECUREVAULT\Domain Admins

PS C:\Users\Administrator.DC01>

```

Get-LapsADPassword output for CLIENT01 (password value redacted)

7.3 – 7.4 — JEA Restricted Session Established

Command(s) executed:

```

Register-PSSessionConfiguration -Path "C:\SecureVaultOperator.pssc" -Name
"SecureVaultOperator" -Force
Enter-PSSession -ComputerName "CLIENT01" -ConfigurationName "SecureVaultOperator" -
Credential (New-Object
    System.Management.Automation.PSCredential("SECUREVAULT\jessy.pinkman",
        (ConvertTo-SecureString "*****" -AsPlainText -Force)))
Get-Command

```

This shows me connecting to CLIENT01 as the restricted user through the registered SecureVaultOperator JEA endpoint, then running Get-Command inside that session. JEA is the free stand-in for CyberArk's PSM (Privileged Session Manager) — it enforces the same least-privilege principle by exposing only a defined, minimal command surface to a connecting user, instead of a full administrative shell. The test account's password is redacted in this screenshot.


```

Administrator: C:\WINDOWS\system32\cmd.exe
WARNING: To launch Server Configuration tool again, run "SConfig"
PS C:\Users\Administrator.SECUREVAULT> powershell
Windows PowerShell
Copyright (c) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator.SECUREVAULT> Enter-PSSession -ComputerName "CLIENT01" -ConfigurationName "SecureVaultOperator" -Cred
ential (New-Object System.Management.Automation.PSCredential("SECUREVAULT\jessy.pinkman"),(ConvertTo-SecureString
-AsPlainText -Force))
[CLIENT01]: PS C:\WINDOWS\system32> Get-Command

CommandType      Name                                     Version      Source
-----
Alias             Add-AppPackage                         2.0.1.0      Appx
Alias             Add-AppPackageVolume                  2.0.1.0      Appx
Alias             Add-AppProvisionedPackage             3.0          Dism
Alias             Add-MsixPackage                       2.0.1.0      Appx
Alias             Add-MsixPackageVolume                 2.0.1.0      Appx
Alias             Add-MsixVolume                       2.0.1.0      Appx
Alias             Add-ProvisionedAppPackage             3.0          Dism
Alias             Add-ProvisionedAppSharedPackageContainer 3.0          Dism
Alias             Add-ProvisionedAppxPackage            3.0          Dism
Alias             Add-WindowsFeature                   2.0.0.0      ServerManager
Alias             Apply-WindowsUnattend                 3.0          Dism
Alias             Disable-PhysicalDiskIndication         2.0.0.0      Storage
Alias             Disable-PhysicalDiskIndication         1.0.0.0      VMDirectStorage
Alias             Disable-StorageDiagnosticLog           2.0.0.0      Storage
Alias             Disable-StorageDiagnosticLog           1.0.0.0      VMDirectStorage
Alias             Dismount-AppPackageVolume              2.0.1.0      Appx
Alias             Dismount-MsixPackageVolume             2.0.1.0      Appx
Alias             Dismount-MsixVolume                   2.0.1.0      Appx
Alias             Enable-PhysicalDiskIndication           2.0.0.0      Storage
Alias             Enable-PhysicalDiskIndication           1.0.0.0      VMDirectStorage
Alias             Enable-StorageDiagnosticLog             2.0.0.0      Storage
Alias             Enable-StorageDiagnosticLog             1.0.0.0      VMDirectStorage
Alias             Expand-IscsiVirtualDisk                 2.0.0.0      IscsiTarget
Alias             Flush-Volume                           2.0.0.0      Storage
Alias             Flush-Volume                           1.0.0.0      VMDirectStorage
Alias             Get-AppPackage                         2.0.1.0      Appx
Alias             Get-AppPackageAutoUpdateSettings        2.0.1.0      Appx
Alias             Get-AppPackageDefaultVolume            2.0.1.0      Appx
Alias             Get-AppPackageLastError                2.0.1.0      Appx
Alias             Get-AppPackageLog                      2.0.1.0      Appx
Alias             Get-AppPackageManifest                 2.0.1.0      Appx
Alias             Get-AppPackageVolume                   2.0.1.0      Appx

```

Enter-PSSession into the SecureVaultOperator JEA endpoint (credential redacted) and Get-Command output

7.4 — JEA Restricted Session — Verifying Allowed Commands

Command(s) executed:

```
Get-Service
```

Still inside the restricted JEA session, running Get-Service succeeds because it was one of the four cmdlets explicitly whitelisted in the SecureVaultOperator role capability file (Get-Service, Restart-Service, Get-Process, Get-EventLog). Anything outside that list — including destructive commands like Remove-Item or Stop-Computer — is simply not visible or callable in this session. This is the concrete, demonstrable proof that the JEA endpoint enforces the same command-filtering control that CyberArk PSM applies to a monitored privileged session.

```

Administrator: C:\WINDOWS\system32\cmd.exe
Cmdlet      Wait-Debugger      3.1.0.0      Microsoft.PowerShell.Utility
Cmdlet      Wait-Event         3.1.0.0      Microsoft.PowerShell.Utility
Cmdlet      Wait-Job           3.0.0.0      Microsoft.PowerShell.Core
Cmdlet      Wait-Process       3.1.0.0      Microsoft.PowerShell.Management
Cmdlet      Where-Object       3.0.0.0      Microsoft.PowerShell.Core
Cmdlet      Write-Debug        3.1.0.0      Microsoft.PowerShell.Utility
Cmdlet      Write-Error        3.1.0.0      Microsoft.PowerShell.Utility
Cmdlet      Write-EventLog     3.1.0.0      Microsoft.PowerShell.Management
Cmdlet      Write-Host         3.1.0.0      Microsoft.PowerShell.Utility
Cmdlet      Write-Information  3.1.0.0      Microsoft.PowerShell.Utility
Cmdlet      Write-Output       3.1.0.0      Microsoft.PowerShell.Utility
Cmdlet      Write-Progress     3.1.0.0      Microsoft.PowerShell.Utility
Cmdlet      Write-Verbose      3.1.0.0      Microsoft.PowerShell.Utility
Cmdlet      Write-Warning      3.1.0.0      Microsoft.PowerShell.Utility

[CLIENT01]: PS C:\WINDOWS\system32> Get-service

Status      Name              DisplayName
-----
Stopped     AppIDSvc          Application Identity
Stopped     AppMgmt           Application Management
Stopped     AppReadiness     App Readiness
Stopped     AppXSvc           AppX Deployment Service (AppXSVC)
Running     BFE               Base Filtering Engine
Stopped     BITS              Background Intelligent Transfer Ser...
Stopped     CertPropSvc       Certificate Propagation
Stopped     ClipSVC           Client License Service (ClipSVC)
Stopped     COMSysApp         COM+ System Application
Running     CoreMessagingRe... CoreMessaging
Running     CryptSvc          Cryptographic Services
Running     DcomLaunch        DCOM Server Process Launcher
Stopped     dcsvc             Declared Configuration(DC) service
Stopped     defragsvc         Optimize drives
Stopped     DeviceInstall     Device Install Service
Running     Dhcp              DHCP Client
Running     DiagTrack         Connected User Experiences and Tele...
Running     DispBrokerDeskt... Display Policy Service
Stopped     dmwappushservice  Device Management Wireless Applicat...
Running     Dnscache          DNS Client
Stopped     DoSvc             Delivery Optimization
Running     DPS              Diagnostic Policy Service
Stopped     EapHost           Extensible Authentication Protocol
Stopped     EFS              Encrypting File System (EFS)
Running     EventLog          Windows Event Log
Running     EventSystem       COM+ Event System

```

Get-Service succeeding inside the restricted JEA session on CLIENT01

8.6 — CONJURVM Docker Containers Running

Command(s) executed:

```
docker compose ps
```

This confirms all five containers that make up the Conjur Open Source stack are up and healthy on CONJURVM: the Conjur server itself, the Postgres database backing it, the Nginx proxy handling TLS, the Conjur CLI client container, and a small bot_app container used for testing. This is the infrastructure layer that hosts the real, open-source CyberArk Conjur secrets engine — the same product CyberArk sells and supports at enterprise scale.

```

conjuradmin@conjurvm:~/conjur-quickstart$ docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED         STATUS         PORTS
bot_app             cfmanteiga/alpine-bash-curl-jq     "tail -F anything"                   bot_app    About an hour ago Up 2 minutes   443/tcp, 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
conjur-quickstart-pgadmin-1 dpape/pgadmin4                      "/entrypoint.sh"                     pgadmin    About an hour ago Up About a minute 443/tcp, 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
conjur_client       cyberark/conjur-cli:9               "sleep infinity"                     client     About an hour ago Up 33 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
conjur_server       cyberark/conjur                     "conjurctl server"                   conjur     About an hour ago Up About a minute 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
nginx_proxy         nginx:latest                         "/docker-entrypoint.sh"              proxy      About an hour ago Up 34 seconds   80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp
postgres_database   postgres:15                         "docker-entrypoint.sh"               database   About an hour ago Up About a minute 0.0.0.0:8432->5432/tcp, [::]:8432->5432/tcp
conjuradmin@conjurvm:~/conjur-quickstart$ _

```

docker compose ps showing the full Conjur Open Source stack running

8.7 – 8.10 — Conjur Account Created, Policy Loaded, and Secret Stored/Retrieved

Command(s) executed:

```

docker compose exec client conjur init -u https://proxy -a securevault --self-signed
docker compose exec client conjur login -i admin
cat << 'EOF' > root.yml
- !policy
  id: securevault-campus
  body:
  - !host
    id: campus-webapp
  - !variable
    id: db-password
  - !permit
    role: !host campus-webapp
    privilege: [ read, execute ]
    resource: !variable db-password
EOF
docker cp root.yml $(docker compose ps -q client):/root.yml
docker compose exec client conjur policy load -b root -f /root.yml
docker compose exec client conjur variable set -i securevault-campus/db-password -v
*****
docker compose exec client conjur variable get -i securevault-campus/db-password

```

This is the core Conjur workflow: I initialized and logged into the Conjur CLI client against the self-signed proxy, authored a policy file that defines a machine identity (!host campus-webapp) and a secret (!variable

db-password)), granted that host read/execute permission on the secret, loaded the policy into Conjur, then set and retrieved the secret value. This is the same application-secrets workflow a real service would use in production — fetching a database credential at runtime through an authenticated API call instead of hard-coding it in source. The actual secret value and API keys are redacted in this screenshot.

```

bqgl3Vate/1Tm/qVUuL1dS4FSz4r2pPPMBN8MHPFOP3NHK7Bsc1k+LhnNtg9xN4J
xVSUimSuaUJmeAbgtMimInAk053GRh05cy4zVREVoJmEzKVd1u/7YrxnHSL2mYrh
80B4V0080p5cEfiYpHwF080Xdk8YtY0g2+vx4u0eRtsc7m0uME005tE3Wic
ULrPDa/akDVYEDAwVzhfCdFvxIfaadm4eTclQE9Tg3WJ1ktJLYfXeIL+32TUHM
JWf+IbBHioY4RshC00zPvznyB1wAUeYnxhDyHkdTpyFHeASE56dyaTJgVMMLpK21
QQIDAQAB
-----END PUBLIC KEY
API key for admin:
conjuradmin@conjurvm:~/conjur-quickstart$ docker compose exec client conjur init -u https://proxy -a securevault --self-signed
Warning: Using self-signed certificates is not recommended and could lead to exposure of sensitive data
Wrote certificate to /home/cli/conjur-server.pem
Wrote configuration to /home/cli/conjurrc
conjuradmin@conjurvm:~/conjur-quickstart$ docker compose exec client conjur login -i admin
Logged in
conjuradmin@conjurvm:~/conjur-quickstart$ cat <<'EOF'> ROOT.YML
> - !POLICY
> - !policy
>   id: securevault-campus
>   body:
>     - !host
>       id: campus-webapp
>     - !variable
>       id: db-password
>     - !permit
>       role: !host campus-webapp
>       privilege: [ read, execute ]
>       resource: !variable db-password
> EOF
conjuradmin@conjurvm:~/conjur-quickstart$ docker cp root.yml $(docker compose ps -q client):/root.yml
Successfully copied 236B (transferred 2.05kB) to cff19c52c64bc043f3f0c7286e7e7a702a07046304cbe827f17b0891a7761e96:/root.yml
conjuradmin@conjurvm:~/conjur-quickstart$ docker compose exec client conjur policy load -b root -f /root.yml
Loaded policy 'root'
{
  "created_roles": {
    "securevault:host:securevault-campus/campus-webapp": {
      "id": "securevault:host:securevault-campus/campus-webapp"
      "api_key":
    }
  },
  "version": 1
}
conjuradmin@conjurvm:~/conjur-quickstart$ docker compose exec client conjur variable set -i securevault-campus/db-password -v
Value added
conjuradmin@conjurvm:~/conjur-quickstart$ docker compose exec client conjur variable get -i securevault-campus/db-password
conjuradmin@conjurvm:~/conjur-quickstart$ [ 2369.988840] watchdog: BUG: soft lockup - CPU#1 stuck for 39s! [containerd-shim:7565]
[ 2370.020885] watchdog: BUG: soft lockup - CPU#0 stuck for 39s! [swapper/0:0]

```

Conjur CLI login, policy load, and secret set/get workflow (secret value and API keys redacted)

8.10 — Conjur Secret Retrieval — Confirmation Run

Command(s) executed:

```

docker compose exec client conjur policy load -b root -f /root.yml
docker compose exec client conjur variable set -i securevault-campus/db-password -v
"*****"
docker compose exec client conjur variable get -i securevault-campus/db-password

```

This second run re-confirms the policy loads cleanly and the secret round-trips correctly after restarting the CONJURVM containers — an important habit for demonstrating that the lab is repeatable and not a one-off fluke. Being able to reload policy and re-verify secret retrieval on demand is exactly the kind of operational reliability check a PAM/secrets-management admin would be expected to perform. The secret value is redacted in this screenshot.

```
conjuradmin@conjurvm:~/conjur-quickstart$ docker compose exec client conjur policy load -b root -f /root.yml
Loaded policy 'root'
{
  "created_roles": {},
  "version": 3
}
conjuradmin@conjurvm:~/conjur-quickstart$ docker compose exec client conjur variable set -i securevault-campus/db-password -v
Value added
conjuradmin@conjurvm:~/conjur-quickstart$ docker compose exec client conjur variable get -i securevault-campus/db-password
conjuradmin@conjurvm:~/conjur-quickstart$ _
```

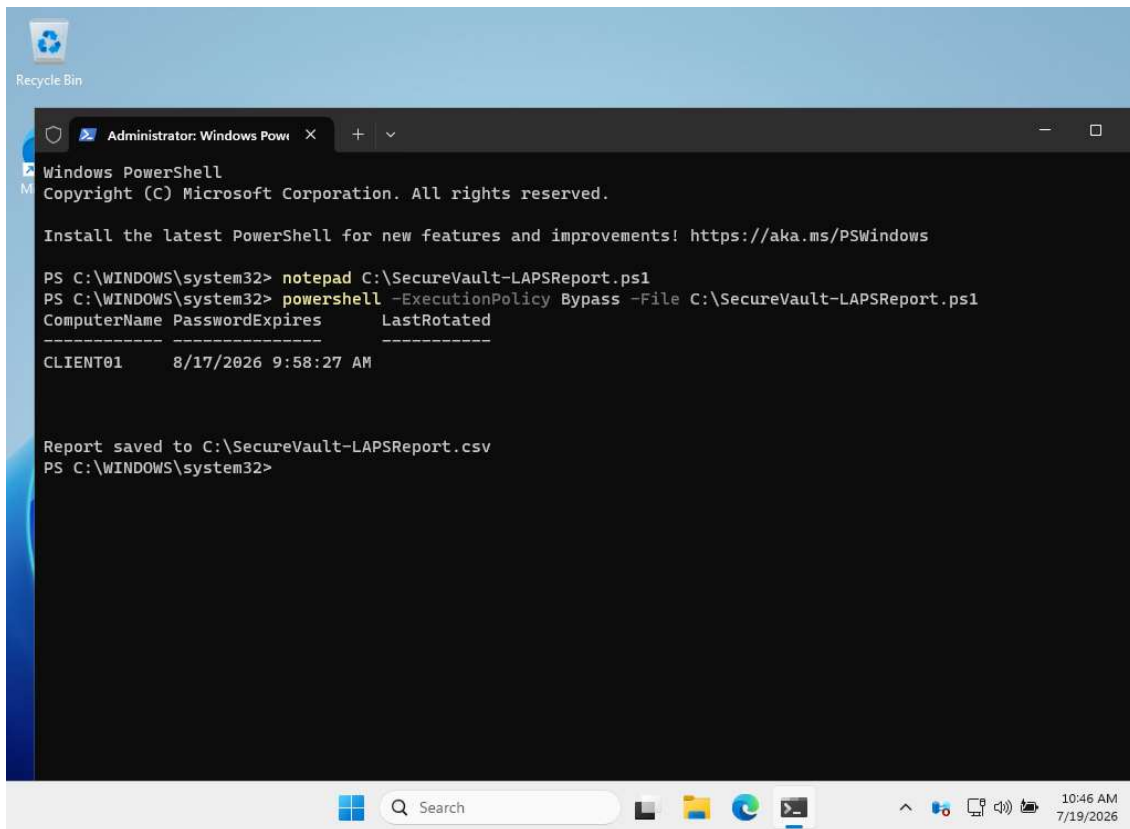
Second confirmation run of Conjur policy load and secret set/get (secret value redacted)

9 — PowerShell LAPS Audit Report

Command(s) executed:

```
notepad C:\SecureVault-LAPSReport.ps1
powershell -ExecutionPolicy Bypass -File C:\SecureVault-LAPSReport.ps1
```

This custom PowerShell script queries every computer object in the Workstations OU, pulls each one's current LAPS password-expiration and last-rotation timestamp, and exports the results to a CSV audit file. The screenshot shows the script running successfully against CLIENT01 and writing SecureVault-LAPSReport.csv. This is my stand-in for CyberArk's built-in reporting/audit capability — a compliance-style report an administrator would pull to prove privileged credentials are rotating on schedule across the environment.



```
Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> notepad C:\SecureVault-LAPSReport.ps1
PS C:\WINDOWS\system32> powershell -ExecutionPolicy Bypass -File C:\SecureVault-LAPSReport.ps1
ComputerName PasswordExpires      LastRotated
-----
CLIENT01      8/17/2026 9:58:27 AM

Report saved to C:\SecureVault-LAPSReport.csv
PS C:\WINDOWS\system32>
```

SecureVault-LAPSReport.ps1 running and exporting a password-rotation audit CSV

4 Challenges Faced and How I Resolved Them

4.1 — Docker Repository / GPG Key Conflicts on Ubuntu

When installing Docker Engine on CONJURVM, the apt repository setup initially failed because of a stale or conflicting GPG key reference from a previous attempt at adding Docker's signing key. I resolved this by removing the old keyring file under `/etc/apt/keyrings` before re-running the install commands, then re-adding Docker's GPG key cleanly with `gpg --dearmor` and re-pointing the apt source list at the correctly signed repository. Running `sudo apt update` afterward confirmed the repository was trusted and `docker-ce` installed without further errors.

4.2 — Running Low on Disk Space Across Three VMs

With only 45 GB free on my D: drive and three VMs to build, I had to actively manage disk usage rather than let it grow unchecked. I addressed this by installing CLIENT01 as Server Core instead of Desktop Experience (cutting its footprint roughly in half), deleting the Windows Server ISO immediately after both Windows VMs were built and before downloading the Ubuntu ISO (so the two large installer files never sat on disk at the same time), and periodically running `Get-PSDrive` to check free space at each major checkpoint. When space ran low, I also used `VBoxManage modifymedium --compact` against each VM's `.vdi` file to reclaim space from deleted data inside the dynamically-allocated disks.

4.3 — LAPS Password Not Appearing Immediately After GPO Link

After linking and configuring the LAPS-Policy GPO, my first Get-LapsADPassword query on CLIENT01 returned nothing. This happened because the Group Policy hadn't been pulled down and applied yet, and the LAPS scheduled task that performs the actual rotation hadn't run since the last policy refresh. I fixed this by running gpupdate /force on CLIENT01 to pull the policy immediately, then restarting CLIENT01 to guarantee the LAPS scheduled task executed on boot. Waiting a couple of minutes after the restart and re-running Get-LapsADPassword confirmed a rotated password was now stored and retrievable.

5 Conclusion and Key Takeaways

Building SecureVault Campus end-to-end — from a bare Active Directory forest through automated password rotation, restricted administrative sessions, and a real CyberArk secrets engine — gave me hands-on exposure to the same architectural problems any PAM deployment has to solve: how do you manage privileged identities, how do you rotate credentials automatically and prove it's happening, how do you restrict what an administrator can do in a session, and how do you let applications fetch secrets securely at runtime instead of hard-coding them.

The biggest takeaway for me was that PAM isn't one product — it's a set of security principles (least privilege, automated rotation, controlled sessions, auditable access) that can be demonstrated with free, genuine tools even without an enterprise CyberArk license. Being able to speak fluently about where my lab uses the real CyberArk product (Conjur) versus where I substituted a functionally equivalent free tool (LAPS for CPM, JEA for PSM) is, I believe, a stronger and more credible story than a black-box claim of "CyberArk experience."

Going forward, I plan to extend this lab with Conjur's dynamic secrets and rotation features, explore integrating it with a small application (so a real service authenticates to Conjur and pulls its database credential at startup), and continue building toward the AWS Certified Cloud Practitioner certification alongside this PAM-focused track so that identity/security and cloud infrastructure skills reinforce each other in my job search.